คู่มือเล่ม 4: Clean Architecture และการแยกชั้น

การออกแบบซอฟต์แวร์ที่แยก Business Logic ออกจากเทคโนโลยี

> ชุดคู่มือสถาปัตยกรรมซอฟต์แวร์และการบริหารโครงการ
> เล่ม 4 จาก 7 เล่ม
> ฉบับสมบูรณ์ v1.0

คำนำ

Clean Architecture เป็นแนวทางที่ Robert C. Martin (Uncle Bob) นำเสนอเพื่อแก้ปัญหาที่พบบ่อยที่สุดของซอฟต์แวร์ คือ **Business Logic ถูกผูกติดกับเทคโนโลยี** จนทำให้ยากต่อการทดสอบ การเปลี่ยนแปลง และการบำรุงรักษา

ปัญหาที่พบบ่อย:

- เปลี่ยน database แล้วต้องแก้ business logic
- เปลี่ยน UI framework แล้วต้องเขียนใหม่
- ทดสอบ business logic ต้องใช้ database จริง
- Business rules กระจัดกระจายใน controller, service, repository

คู่มือเล่มนี้มีเป้าหมายเพื่อให้ผู้อ่าน **เข้าใจหลักการของ Clean Architecture อย่างลึกซึ้ง** ตั้งแต่ Dependency Rule, การแยกชั้นทั้งสี่ (Entities, Use Cases, Interface Adapters, Frameworks & Drivers), วิธีทดสอบ, วิธีเปลี่ยนเทคโนโลยีโดยกระทบน้อยที่สุด ไปจนถึงการประยุกต์ใช้ในองค์กร

คู่มือเล่มนี้ต่อยอดจากเล่ม 3 (DDD) โดยเฉพาะการนำโมเดลโดเมนมาวางในชั้น Entities และ Use Cases

---

ข้อมูลเอกสาร

รายการ	รายละเอียด
ชื่อเล่ม	Clean Architecture และการแยกชั้น
ชื่อรอง	การออกแบบซอฟต์แวร์ที่แยก Business Logic ออกจากเทคโนโลยี
กลุ่มผู้อ่าน	Software Architect, Tech Lead, Senior Developer, Tester, DevOps

รายการ	รายละเอียด
ระดับ	กลางถึงสูง
โครงสร้างหลัก	เรื่อง → วัตถุประสงค์ → เป้าหมาย → ข้อดี → ข้อเสีย → ข้อควรระวัง → ตัวอย่าง → การประยุกต์ใช้ → สรุป
เล่มที่เชื่อมโยง	เล่ม 1 (พื้นฐาน), เล่ม 3 (DDD), เล่ม 5 (ความเสี่ยง), เล่ม 6 (POC), เล่ม 7 (การบริหารโครงการ)
จำนวนบท	17 บท + 7 ภาคผนวก

วิธีใช้เอกสาร

1. **บทที่ 1–4** ใช้สร้างความเข้าใจพื้นฐานและหลักการ — อ่านก่อนทุกบท
2. **บทที่ 5–8** เจาะลึกแต่ละชั้น — อ่านก่อนออกแบบ
3. **บทที่ 9–12** หลักการและแนวปฏิบัติ — อ่านก่อนลงมือเขียนโค้ด
4. **บทที่ 13–14** การทดสอบและการเปลี่ยนเทคโนโลยี — อ่านก่อนวางแผน
5. **บทที่ 15–17** กรณีศึกษาและการประยุกต์ใช้ — อ่านก่อนสรุปแผน
6. **ภาคผนวก** ใช้เป็นเครื่องมือประกอบการออกแบบและ review

โครงสร้างบทมาตรฐาน

ทุกบทในเล่มนี้ใช้โครงสร้างร่วมกันดังนี้

1. เรื่อง
2. วัตถุประสงค์
3. เป้าหมาย
4. หัวข้อหลัก
5. ข้อดีของบท
6. ข้อเสีย/ข้อจำกัด
7. ข้อควรระวัง
8. ตัวอย่าง
9. การประยุกต์ใช้
10. สรุป/ผลลัพธ์
11. เชื่อมโยงกับเล่มอื่น

สารบัญ

ส่วนที่ 1: ภาพรวมและหลักการพื้นฐาน

- บทที่ 1 Clean Architecture คืออะไร
- บทที่ 2 ปัญหาที่ Clean Architecture แก้
- บทที่ 3 Dependency Rule: หัวใจของ Clean Architecture
- บทที่ 4 ภาพรวมสี่ชั้นและความสัมพันธ์

ส่วนที่ 2: สี่ชั้นของ Clean Architecture

- บทที่ 5 ชั้น Entities
- บทที่ 6 ชั้น Use Cases
- บทที่ 7 ชั้น Interface Adapters
- บทที่ 8 ชั้น Frameworks & Drivers

ส่วนที่ 3: หลักการและแนวปฏิบัติ

- บทที่ 9 การแยก Business Logic ออกจาก Database
- บทที่ 10 การแยก Business Logic ออกจาก Framework
- บทที่ 11 การแยก Business Logic ออกจาก UI
- บทที่ 12 Dependency Injection และ Inversion of Control

ส่วนที่ 4: การทดสอบและการเปลี่ยนเทคโนโลยี

- บทที่ 13 วิธีทดสอบ Clean Architecture
- บทที่ 14 วิธีเปลี่ยนเทคโนโลยีโดยกระทบน้อยที่สุด

ส่วนที่ 5: การนำไปใช้และกรณีศึกษา

- บทที่ 15 ข้อดี/ข้อเสียของ Clean Architecture
- บทที่ 16 ตัวอย่างการออกแบบระบบด้วย Clean Architecture
- บทที่ 17 การประยุกต์ใช้และสรุปเล่ม

ภาคผนวก

- ภาคผนวก A: Layer Dependency Checklist
- ภาคผนวก B: Use Case Template

- ภาคผนวก C: Interface Adapter Checklist
- ภาคผนวก D: Test Strategy Template
- ภาคผนวก E: Technology Migration Checklist
- ภาคผนวก F: Code Structure Template
- ภาคผนวก G: Glossary เฉพาะเล่ม 4

ส่วนที่ 1: ภาพรวมและหลักการพื้นฐาน

บทที่ 1: Clean Architecture คืออะไร

1.1 เรื่อง

ความหมาย ที่มา และหลักการพื้นฐานของ Clean Architecture

1.2 วัตถุประสงค์

- สร้างคำนิยามร่วมของ Clean Architecture
- อธิบายที่มาและวิวัฒนาการ
- ทำให้เห็นภาพรวมก่อนลงรายละเอียด

1.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- อธิบาย Clean Architecture ด้วยคำของตนเอง
- เข้าใจหลักการพื้นฐาน
- เห็นความเชื่อมโยงกับ Hexagonal และ Onion

1.4 หัวข้อหลัก

1.4.1 ความหมายของ Clean Architecture

Clean Architecture คือ **แนวทางจัดโครงสร้างซอฟต์แวร์** ที่:

- แยก **Business Logic** ออกจาก **เทคโนโลยี** (database, framework, UI)
- ใช้ **Dependency Rule** เป็นหลักการหลัก
- จัดชั้นเป็นวงกลมซ้อนกัน (concentric circles)
- ให้ **Entities** อยู่ศูนย์กลาง
- ให้ **Frameworks & Drivers** อยู่ชั้นนอกสุด

1.4.2 ที่มาและวิวัฒนาการ

Clean Architecture ไม่ได้เกิดขึ้นลอย ๆ แต่เป็นการรวมแนวคิดหลายอย่าง:

ปี	แนวคิด	ผู้เสนอ
2005	Hexagonal Architecture (Ports & Adapters)	Alistair Cockburn
2008	Onion Architecture	Jeffrey Palermo
2012	Screaming Architecture	Robert C. Martin
2012	BCE (Boundary-Control-Entity)	Ivar Jacobson
2017	Clean Architecture	Robert C. Martin

ทั้งหมดมีหลักการร่วม: **แยก core business logic ออกจาก infrastructure**

1.4.3 หลักการพื้นฐาน

1. **Independent of Frameworks** — framework เป็นเครื่องมือ ไม่ใช่ข้อจำกัด
2. **Testable** — business logic ทดสอบได้โดยไม่ต้องใช้ database/UI
3. **Independent of UI** — เปลี่ยน UI ได้โดยไม่กระทบ business logic
4. **Independent of Database** — เปลี่ยน database ได้โดยไม่กระทบ business logic
5. **Independent of External Agencies** — business logic ไม่รู้จักโลกภายนอก

1.4.4 Clean Architecture vs Hexagonal vs Onion

ลักษณะ	Hexagonal	Onion	Clean
ศูนย์กลาง	Domain	Domain Model	Entities
ชั้น	Ports & Adapters	หลายชั้น	4 ชั้น
Dependency Rule	มี	มี	มี (ชัดเจนที่สุด)
Use Case	ไม่ชัด	มี	มี (ชัดเจน)
ความซับซ้อน	กลาง	กลาง	สูง

1.4.5 สิ่งที่ Clean Architecture ไม่ใช่

- ไม่ใช่ framework
- ไม่ใช่ library
- ไม่ใช่ silver bullet
- ไม่ใช่ "ต้องมี 4 ชั้นเป๊ะ ๆ"

- ไม่ใช่ “ต้องใช้ Dependency Injection เสมอ”

1.5 ข้อดีของบท

- สร้างความเข้าใจพื้นฐานที่ถูกต้อง
- ทำให้เห็นที่มาและความสัมพันธ์
- เตรียมพื้นฐานสำหรับบทถัดไป

1.6 ข้อเสีย/ข้อจำกัด

- เป็นภาพรวม ยังไม่ลงรายละเอียด
- ตัวอย่างเป็นระดับแนวคิด

1.7 ข้อควรระวัง

- อย่าคิดว่า Clean Architecture คือคำตอบทุกปัญหา
- อย่าละเลยความซับซ้อนที่เพิ่มขึ้น
- อย่าใช้โดยไม่เข้าใจหลักการ

1.8 ตัวอย่าง

ระบบที่มีปัญหา:

- Business logic อยู่ใน controller
- Business logic เรียก database โดยตรง
- เปลี่ยน database แล้วต้องแก้หลายที่
- ทดสอบ business logic ต้องใช้ database จริง

Clean Architecture แก้ปัญหาเหล่านี้โดยแยก business logic ไว้ที่ Entities และ Use Cases

1.9 การประยุกต์ใช้

- ใช้เป็นจุดเริ่มต้นในการทำความเข้าใจ
- ใช้ประเมินว่าระบบเหมาะกับ Clean Architecture หรือไม่
- ใช้สื่อสารกับทีม

1.10 สรุป/ผลลัพธ์

ผู้อ่านเข้าใจความหมายและหลักการของ Clean Architecture

1.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 2–4
- ไปสู่เล่ม 3 (DDD — Entities และ Use Cases)
- ไปสู่เล่ม 1 (Hexagonal, Onion)

บทที่ 2: ปัญหาที่ Clean Architecture แก้

2.1 เรื่อง

ปัญหาที่พบบ่อยในซอฟต์แวร์ที่ Clean Architecture ออกแบบมาเพื่อแก้

2.2 วัตถุประสงค์

- ทำให้เห็นปัญหาที่ชัดเจน
- เชื่อมปัญหาเข้ากับหลักการ
- เตรียมพื้นฐานสำหรับการแก้

2.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- ระบุปัญหาในระบบของตนเองได้
- เข้าใจว่าปัญหาเกิดจากอะไร
- เห็นว่า Clean Architecture แก้อย่างไร

2.4 หัวข้อหลัก

2.4.1 ปัญหาที่ 1: Business Logic กระจัดกระจาย

อาการ:

- Business rules อยู่ใน controller
- Validation อยู่ใน service
- Calculation อยู่ใน repository
- ไม่มีที่เดียวที่บอกว่า business logic คืออะไร

ผลกระทบ:

- แก้ business rule ต้องแก้หลายที่
- ทดสอบยาก
- เข้าใจยาก

2.4.2 ปัญหาที่ 2: ผูกติดกับ Database

อาการ:

- Business logic เรียก SQL โดยตรง
- Entity มี annotation ของ ORM
- เปลี่ยน database ต้องแก้ business logic

ผลกระทบ:

- ทดสอบต้องใช้ database จริง
- เปลี่ยน database ยาก
- Business logic ไม่ independent

2.4.3 ปัญหาที่ 3: ผูกติดกับ Framework

อาการ:

- Business logic สืบทอดจาก framework class
- Business logic ใช้ framework annotation
- เปลี่ยน framework ต้องเขียนใหม่

ผลกระทบ:

- ไม่สามารถทดสอบได้โดยไม่ใช้ framework
- เปลี่ยน framework ยาก
- Business logic ไม่ portable

2.4.4 ปัญหาที่ 4: ผูกติดกับ UI

อาการ:

- Business logic อยู่ใน controller/view
- Business logic ใช้ HTTP request/response
- เปลี่ยน UI ต้องแก้ business logic

ผลกระทบ:

- ไม่สามารถใช้ business logic กับ UI อื่นได้
- ทดสอบยาก
- Business logic ไม่ independent

2.4.5 ปัญหาที่ 5: ทดสอบยาก

อาการ:

- ทดสอบ business logic ต้องใช้ database
- ทดสอบ business logic ต้องใช้ web server
- ทดสอบช้า

ผลกระทบ:

- ทดสอบไม่ครอบคลุม
- Bug หลุดไป production
- พัฒนาช้า

2.4.6 ตัวอย่างปัญหาในโค้ด

```java
// ปัญหา: Business logic อยู่ใน controller
@RestController
public class OrderController {
    @Autowired
    private JdbcTemplate jdbc;

    @PostMapping("/orders")
    public ResponseEntity<?> createOrder(@RequestBody OrderRequest req) {
        // Business logic อยู่ใน controller
        if (req.getItems().isEmpty()) {
            return ResponseEntity.badRequest().build();
        }

        // เรียก database โดยตรง
        BigDecimal total = BigDecimal.ZERO;
        for (OrderItem item : req.getItems()) {
            BigDecimal price = jdbc.queryForObject(
                "SELECT price FROM products WHERE id = ?",
                BigDecimal.class, item.getProductId());
            total = total.add(price.multiply(item.getQuantity()));
        }

        // SQL ตรง ๆ
        jdbc.update("INSERT INTO orders ...");

        return ResponseEntity.ok().build();
    }
}
```

2.4.7 Clean Architecture แก้อย่างไร

- แยก business logic ไปที่ **Entities** และ **Use Cases**
- ใช้ **Repository interface** แทนการเรียก database โดยตรง
- ใช้ **Dependency Rule** ทำให้ business logic ไม่รู้จัก infrastructure
- ใช้ **Interface Adapters** แปลงข้อมูลระหว่างชั้น

2.5 ข้อดีของบท

- ทำให้เห็นปัญหาที่ชัดเจน
- เชื่อมปัญหาเข้ากับหลักการ
- เตรียมพื้นฐานสำหรับการแก้

2.6 ข้อเสีย/ข้อจำกัด

- ตัวอย่างอาจไม่ครอบคลุมทุกกรณี
- ปัญหาบางอย่างอาจต้องใช้แนวทางอื่น

2.7 ข้อควรระวัง

- อย่าละเลยปัญหาเล็ก ๆ ที่สะสม
- อย่ารอให้ปัญหาใหญ่ก่อนแก้
- อย่าแก้ปัญหาด้วยการเพิ่มความซับซ้อนโดยไม่จำเป็น

2.8 ตัวอย่าง

ระบบที่มีปัญหา:

- OrderController มี business logic
- OrderService เรียก JdbcTemplate โดยตรง
- Order entity มี @Entity annotation
- เปลี่ยน database จาก MySQL เป็น PostgreSQL ต้องแก้หลายไฟล์

2.9 การประยุกต์ใช้

- ใช้ระบุปัญหาในระบบของตนเอง
- ใช้สื่อสารกับทีม
- ใช้เป็นแรงจูงใจในการ refactor

2.10 สรุป/ผลลัพธ์

ผู้อ่านระบุปัญหาในระบบของตนเองได้ และเข้าใจว่า Clean Architecture แก้อย่างไร

2.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 3–4
- ไปสู่เล่ม 3 (DDD — การแยก Business Logic)

บทที่ 3: Dependency Rule: หัวใจของ Clean Architecture

3.1 เรื่อง

Dependency Rule — หลักการที่สำคัญที่สุดของ Clean Architecture

3.2 วัตถุประสงค์

- อธิบาย Dependency Rule อย่างละเอียด
- ทำให้เห็นว่าทำไมจึงสำคัญ
- เตรียมพื้นฐานสำหรับการออกแบบ

3.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- อธิบาย Dependency Rule ได้
- ใช้ Dependency Rule ในการออกแบบ
- ตรวจสอบว่าระบบปฏิบัติตามหรือไม่

3.4 หัวข้อหลัก

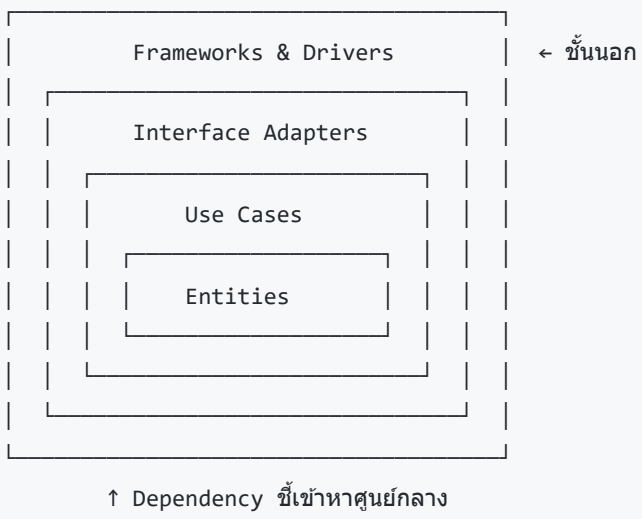
3.4.1 Dependency Rule คืออะไร

Dependency Rule: “Source code dependencies must point only inward, toward higher-level policies”

หรือพูดง่าย ๆ:

- **ชั้นในไม่รู้จักชั้นนอก**
- **Entities ไม่รู้จัก Use Cases**
- **Use Cases ไม่รู้จัก Interface Adapters**
- **Interface Adapters ไม่รู้จัก Frameworks & Drivers**

3.4.2 ทิศทางของ Dependency



3.4.3 ตัวอย่างการละเมิด Dependency Rule

```
// ละเมิด: Entity รู้จัก JPA
@Entity  // JPA annotation — มาจาก infrastructure
public class Order {
    @Id
    private Long id;

    @Column
    private String status;

    // Business logic ที่ผูกกับ JPA
    @PrePersist
    void onCreate() {
        this.createdAt = LocalDateTime.now();
    }
}
```

ปัญหา: Entity รู้จัก JPA ซึ่งเป็น infrastructure

3.4.4 วิธีปฏิบัติตาม Dependency Rule

```
// Entity — ไม่รู้จัก JPA
public class Order {
    private final OrderId id;
    private OrderStatus status;

    public void confirm() {
        if (status != OrderStatus.DRAFT) {
            throw new IllegalStateException("Cannot confirm");
        }
        this.status = OrderStatus.CONFIRMED;
    }
}

// Infrastructure — รู้จัก JPA
@Entity
@Table(name = "orders")
public class OrderJpaEntity {
    @Id
    private String id;

    @Column
    private String status;

    // แปลงไป/กลับ
```

```
    public Order toDomain() { ... }
    public static OrderJpaEntity fromDomain(Order order) { ... }
}
```

3.4.5 Dependency Inversion

เมื่อชั้นในต้องใช้บริการจากชั้นนอก ใช้ **Dependency Inversion**:

- ชั้นในกำหนด **interface**
- ชั้นนอก **implement** interface
- Dependency ยังชี้เข้าหาชั้นใน

ตัวอย่าง:

```
// Use Case — กำหนด interface
public interface OrderRepository {
    Optional<Order> findById(OrderId id);
    void save(Order order);
}

// Use Case — ใช้ interface
public class PlaceOrderUseCase {
    private final OrderRepository orders;  // interface

    public PlaceOrderUseCase(OrderRepository orders) {
        this.orders = orders;
    }

    public OrderId execute(PlaceOrderCommand cmd) {
        Order order = Order.create(cmd.customerId(), cmd.items());
        orders.save(order);
        return order.id();
    }
}

// Infrastructure — implement interface
public class JpaOrderRepository implements OrderRepository {
    // implementation
}
```

3.4.6 การตรวจสอบ Dependency Rule

- **Static analysis:** ใช้ ArchUnit, NDepend
- **Code review:** ตรวจ import
- **Package structure:** ตรวจว่า dependency ชี้เข้าหรือไม่

3.5 ข้อดีของบท

- ทำให้เห็นหลักการที่สำคัญที่สุด
- ให้แนวทางปฏิบัติ
- เตรียมพื้นฐานสำหรับการออกแบบ

3.6 ข้อเสีย/ข้อจำกัด

- ต้องใช้ความพยายามในการรักษา
- อาจต้องใช้เครื่องมือช่วย

3.7 ข้อควรระวัง

- อย่าละเมิด Dependency Rule แม้แต่เล็กน้อย
- อย่าคิดว่า annotation ของ framework เป็นเรื่องเล็ก
- อย่าละเลยการตรวจสอบ

3.8 ตัวอย่าง

ก่อน:

```
// Entity ผูกกับ JPA
@Entity
public class Order { ... }

// Use Case เรียก JPA โดยตรง
public class PlaceOrderUseCase {
    @Autowired
    private JdbcTemplate jdbc;  // ผูกกับ infrastructure

    public void execute(...) {
        jdbc.update("INSERT INTO orders ...");
    }
}
```

หลัง:

```
// Entity ไม่ผูกกับ JPA
public class Order { ... }

// Use Case ใช้ interface
public class PlaceOrderUseCase {
    private final OrderRepository orders;  // interface

    public void execute(...) {
```

```
        orders.save(order);
    }
}

// Infrastructure implement
public class JpaOrderRepository implements OrderRepository { ... }
```

3.9 การประยุกต์ใช้

- ใช้ Dependency Rule Checklist (ภาคผนวก A)
- ใช้ใน code review
- ใช้ใน architecture review

3.10 สรุป/ผลลัพธ์

ผู้อ่านอธิบาย Dependency Rule ได้ และใช้ในการออกแบบ

3.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 4–8
- ไปสู่เล่ม 3 (Dependency Rule ใน Hexagonal)

บทที่ 4: ภาพรวมสี่ชั้นและความสัมพันธ์

4.1 เรื่อง

ภาพรวมของสี่ชั้นใน Clean Architecture และความสัมพันธ์ระหว่างชั้น

4.2 วัตถุประสงค์

- อธิบายแต่ละชั้นโดยสรุป
- ทำให้เห็นความสัมพันธ์
- เตรียมพื้นฐานสำหรับส่วนที่ 2

4.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- อธิบายแต่ละชั้นได้
- เห็นความสัมพันธ์ระหว่างชั้น
- เข้าใจบทบาทของแต่ละชั้น

4.4 หัวข้อหลัก

4.4.1 ชั้น Entities

- **ศูนย์กลาง:** Business rules ระดับองค์กร
- **ไม่มี:** dependency กับชั้นอื่น
- **มี:** business logic ที่สำคัญที่สุด
- **ตัวอย่าง:** Order, Customer, Money

4.4.2 ชั้น Use Cases

- **บทบาท:** Application-specific business rules
- **Orchestrate:** Entities
- **กำหนด:** interface สำหรับ infrastructure
- **ตัวอย่าง:** PlaceOrderUseCase, CalculatePriceUseCase

4.4.3 ชั้น Interface Adapters

- **บทบาท:** แปลงข้อมูลระหว่าง Use Cases กับ Frameworks
- **มี:** Controller, Presenter, Gateway
- **แปลง:** Request → Use Case Input, Use Case Output → Response
- **ตัวอย่าง:** OrderController, OrderPresenter

4.4.4 ชั้น Frameworks & Drivers

- **บทบาท:** เทคโนโลยี (database, web, framework)
- **มี:** JPA, Spring, React, PostgreSQL
- **ไม่มี:** business logic
- **ตัวอย่าง:** JpaOrderRepository, Spring Boot, React Component

4.4.5 ความสัมพันธ์ระหว่างชั้น

```
Frameworks & Drivers
    ↓ depends on
Interface Adapters
    ↓ depends on
Use Cases
    ↓ depends on
Entities
```

Dependency ชี้เข้าหาศูนย์กลางเสมอ

4.4.6 การไหลของข้อมูล

```
Request → Controller → Use Case → Entity
                                     ↓
Response ← Presenter ← Use Case Output
```

4.4.7 ตัวอย่าง

ระบบ e-commerce:

- **Entities:** Order, OrderItem, Money, Customer
- **Use Cases:** PlaceOrderUseCase, CalculatePriceUseCase
- **Interface Adapters:** OrderController, OrderPresenter, OrderRepositoryImpl
- **Frameworks & Drivers:** Spring Boot, JPA, PostgreSQL

4.5 ข้อดีของบท

- ทำให้เห็นภาพรวมของสี่ชั้น
- เตรียมพื้นฐานสำหรับส่วนที่ 2
- ลดความสับสน

4.6 ข้อเสีย/ข้อจำกัด

- เป็นภาพรวม ยังไม่ลงรายละเอียด
- ต้องอ่านส่วนที่ 2 เพื่อความเข้าใจเต็ม

4.7 ข้อควรระวัง

- อย่าคิดว่าต้องมี 4 ชั้นเป๊ะ ๆ
- อย่าละเลย Dependency Rule
- อย่าให้ชั้นนอกมี business logic

4.8 ตัวอย่าง

```
src/
├── domain/                    # Entities
│   ├── Order.java
│   ├── OrderItem.java
│   └── Money.java
├── usecase/                   # Use Cases
│   ├── PlaceOrderUseCase.java
│   └── OrderRepository.java   # interface
├── adapter/                   # Interface Adapters
│   ├── controller/
```

```
│   │   └── OrderController.java
│   ├── presenter/
│   │   └── OrderPresenter.java
│   └── gateway/
│       └── JpaOrderRepository.java
└── infrastructure/            # Frameworks & Drivers
    ├── SpringBootApplication.java
    └── config/
        └── DatabaseConfig.java
```

4.9 การประยุกต์ใช้

- ใช้เป็นแผนที่นำทางก่อนอ่านส่วนที่ 2
- ใช้สื่อสารกับทีม
- ใช้ใน architecture review

4.10 สรุป/ผลลัพธ์

ผู้อ่านเห็นภาพรวมของสี่ชั้นและความสัมพันธ์

4.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่ส่วนที่ 2 (สี่ชั้น)
- ไปสู่เล่ม 3 (DDD — Entities)

ส่วนที่ 2: สี่ชั้นของ Clean Architecture

บทที่ 5: ชั้น Entities

5.1 เรื่อง

ชั้น Entities — Business rules ระดับองค์กร

5.2 วัตถุประสงค์

- อธิบายบทบาทของ Entities
- ทำให้เห็นว่า Entities ควรมีอะไร
- เตรียมพื้นฐานสำหรับ Use Cases

5.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- ออกแบบ Entities ได้
- แยก Entities จาก Use Cases
- รักษา Entities ให้ independent

5.4 หัวข้อหลัก

5.4.1 Entities คืออะไร

Entities คือ **Business rules ระดับองค์กร** ที่:

- เป็นจริงไม่ว่าจะใช้ระบบอย่างไร
- ไม่ขึ้นกับ application ใด ๆ
- ไม่ขึ้นกับ framework, database, UI
- เป็น pure business logic

5.4.2 Entities ควรมีอะไร

- **Business rules** — กฎทางธุรกิจ
- **Validation** — การตรวจสอบความถูกต้อง
- **Calculation** — การคำนวณ
- **State transitions** — การเปลี่ยนสถานะ
- **Invariants** — สิ่งที่ต้องเป็นจริงเสมอ

5.4.3 Entities ไม่ควรมีอะไร

- **Framework annotation** — @Entity, @Component
- **Database logic** — SQL, JPA
- **HTTP logic** — Request, Response
- **External service calls** — API calls
- **Serialization** — JSON, XML

5.4.4 ตัวอย่าง Entities

```
// Entities – pure business logic
public class Order {
    private final OrderId id;
    private final CustomerId customerId;
    private final List<OrderItem> items;
    private OrderStatus status;

    public Order(OrderId id, CustomerId customerId, List<OrderItem> items) {
        if (items == null || items.isEmpty()) {
```

```
            throw new IllegalArgumentException("Order must have items");
        }
        this.id = id;
        this.customerId = customerId;
        this.items = new ArrayList<>(items);
        this.status = OrderStatus.DRAFT;
    }

    public void confirm() {
        if (status != OrderStatus.DRAFT) {
            throw new IllegalStateException("Only draft orders can be confirmed");
        }
        if (items.isEmpty()) {
            throw new IllegalStateException("Cannot confirm empty order");
        }
        this.status = OrderStatus.CONFIRMED;
    }

    public Money total() {
        return items.stream()
            .map(OrderItem::subtotal)
            .reduce(Money.ZERO, Money::add);
    }

    public void addItem(Product product, int quantity) {
        if (status != OrderStatus.DRAFT) {
            throw new IllegalStateException("Cannot modify confirmed order");
        }
        items.add(new OrderItem(product, quantity));
    }
}
```

5.4.5 Entities กับ DDD

- Entities ใน Clean Architecture ≈ Entities/Aggregates ใน DDD
- ใช้ ubiquitous language
- ใช้ value objects
- รักษา invariants

5.4.6 การทดสอบ Entities

- Unit test โดยตรง
- ไม่ต้องใช้ database, framework
- เร็ว, ครอบคลุม
- เป็น test ที่สำคัญที่สุด

5.4.7 ข้อผิดพลาดที่พบบ่อย

- Entity ที่มี annotation ของ framework
- Entity ที่เรียก database
- Entity ที่มี business logic น้อยเกินไป (anemic)
- Entity ที่ใหญ่เกินไป

5.5 ข้อดีของบท

- ทำให้เห็นบทบาทของ Entities
- เตรียมพื้นฐานสำหรับ Use Cases
- สอนการออกแบบ Entities ที่ดี

5.6 ข้อเสีย/ข้อจำกัด

- ต้องอาศัยความเข้าใจโดเมน
- ต้องฝึกฝน

5.7 ข้อควรระวัง

- อย่าให้ Entity ผูกกับ framework
- อย่าให้ Entity เรียก database
- อย่าให้ Entity เป็น anemic

5.8 ตัวอย่าง

```
// Entity ที่ดี
public class Money {
    private final BigDecimal amount;
    private final Currency currency;

    public Money(BigDecimal amount, Currency currency) {
        if (amount == null || currency == null) {
            throw new IllegalArgumentException("Amount and currency required");
        }
        this.amount = amount;
        this.currency = currency;
    }

    public Money add(Money other) {
        if (!this.currency.equals(other.currency)) {
            throw new IllegalArgumentException("Cannot add different currencies");
        }
        return new Money(this.amount.add(other.amount), this.currency);
    }
```

```
    public boolean isGreaterThan(Money other) {
        if (!this.currency.equals(other.currency)) {
            throw new IllegalArgumentException("Cannot compare different currencies");
        }
        return this.amount.compareTo(other.amount) > 0;
    }
}
```

5.9 การประยุกต์ใช้

- ใช้ออกแบบ Entities ในระบบ
- ใช้ใน code review
- ใช้ทดสอบ business logic

5.10 สรุป/ผลลัพธ์

ผู้อ่านออกแบบ Entities ได้ และแยกจาก Use Cases

5.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 6
- ไปสู่เล่ม 3 (Entities ใน DDD)

บทที่ 6: ชั้น Use Cases

6.1 เรื่อง

ชั้น Use Cases — Application-specific business rules

6.2 วัตถุประสงค์

- อธิบายบทบาทของ Use Cases
- ทำให้เห็นว่า Use Cases ควรมีอะไร
- เตรียมพื้นฐานสำหรับ Interface Adapters

6.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- ออกแบบ Use Cases ได้
- แยก Use Cases จาก Entities
- กำหนด interface สำหรับ infrastructure

6.4 หัวข้อหลัก

6.4.1 Use Cases คืออะไร

Use Cases คือ **Application-specific business rules** ที่:

- Orchestrate Entities
- กำหนด flow ของ application
- กำหนด interface สำหรับ infrastructure
- ไม่มี business logic ที่เป็นของ Entities

6.4.2 Use Cases ควรมีอะไร

- **Orchestration** — เรียก Entities ตามลำดับ
- **Transaction boundary** — กำหนดขอบเขต transaction
- **Interface definition** — กำหนด interface สำหรับ infrastructure
- **Input/Output** — กำหนด input/output ของ use case
- **Error handling** — จัดการ error

6.4.3 Use Cases ไม่ควรมีอะไร

- **Business logic** — ควรอยู่ใน Entities
- **Framework annotation** — @Service, @Component
- **Database logic** — SQL, JPA
- **HTTP logic** — Request, Response

6.4.4 ตัวอย่าง Use Case

```
// Use Case
public class PlaceOrderUseCase {
    private final OrderRepository orders;
    private final ProductRepository products;
    private final EventPublisher events;

    public PlaceOrderUseCase(OrderRepository orders,
                             ProductRepository products,
                             EventPublisher events) {
        this.orders = orders;
        this.products = products;
        this.events = events;
    }

    public OrderId execute(PlaceOrderCommand cmd) {
        // Orchestrate Entities
        List<OrderItem> items = cmd.items().stream()
            .map(i -> new OrderItem(
```

```
                products.findById(i.productId())
                    .orElseThrow(() -> new ProductNotFound(i.productId())),
                i.quantity()))
            .toList();

        Order order = new Order(
            OrderId.generate(),
            cmd.customerId(),
            items
        );

        orders.save(order);
        events.publish(new OrderPlaced(order.id(), order.total()));

        return order.id();
    }
}
```

6.4.5 Input/Output ของ Use Case

- **Input:** Command/DTO
- **Output:** Result/DTO
- ไม่ใช้ Entity เป็น input/output โดยตรง (เพื่อแยกชั้น)
- ใช้ interface สำหรับ infrastructure

6.4.6 Use Cases กับ DDD Application Services

- Use Cases ≈ Application Services ใน DDD
- ทั้งสอง orchestrate domain
- ทั้งสองกำหนด transaction boundary
- ทั้งสองไม่เก็บ business logic

6.4.7 การทดสอบ Use Cases

- Unit test โดยใช้ mock repository
- ไม่ต้องใช้ database
- ทดสอบ flow ของ use case

6.4.8 ข้อผิดพลาดที่พบบ่อย

- Use Case ที่มี business logic
- Use Case ที่เรียก database โดยตรง
- Use Case ที่ผูกกับ framework
- Use Case ที่ใหญ่เกินไป

6.5 ข้อดีของบท

- ทำให้เห็นบทบาทของ Use Cases
- เตรียมพื้นฐานสำหรับ Interface Adapters
- สอนการออกแบบ Use Cases ที่ดี

6.6 ข้อเสีย/ข้อจำกัด

- ต้องอาศัยความเข้าใจ Entities
- ต้องฝึกฝน

6.7 ข้อควรระวัง

- อย่าให้ Use Case มี business logic
- อย่าให้ Use Case เรียก database โดยตรง
- อย่าให้ Use Case ผูกกับ framework

6.8 ตัวอย่าง

```
// Input
public record PlaceOrderCommand(
    CustomerId customerId,
    List<OrderItemInput> items
) {}

public record OrderItemInput(ProductId productId, int quantity) {}

// Output
public record PlaceOrderResult(OrderId orderId, Money total) {}

// Use Case
public class PlaceOrderUseCase {
    public PlaceOrderResult execute(PlaceOrderCommand cmd) {
        // ...
        return new PlaceOrderResult(order.id(), order.total());
    }
}
```

6.9 การประยุกต์ใช้

- ใช้ออกแบบ Use Cases ในระบบ
- ใช้ใน code review
- ใช้ทดสอบ application logic

6.10 สรุป/ผลลัพธ์

ผู้อ่านออกแบบ Use Cases ได้ และแยกจาก Entities

6.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 7
- ไปสู่เล่ม 3 (Application Services)

บทที่ 7: ชั้น Interface Adapters

7.1 เรื่อง

ชั้น Interface Adapters — แปลงข้อมูลระหว่าง Use Cases กับ Frameworks

7.2 วัตถุประสงค์

- อธิบายบทบาทของ Interface Adapters
- ทำให้เห็นว่า Adapters ควรมีอะไร
- เตรียมพื้นฐานสำหรับ Frameworks & Drivers

7.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- ออกแบบ Interface Adapters ได้
- แปลงข้อมูลระหว่างชั้น
- รักษา Dependency Rule

7.4 หัวข้อหลัก

7.4.1 Interface Adapters คืออะไร

Interface Adapters คือชั้นที่:

- **แปลงข้อมูล** ระหว่าง Use Cases กับ Frameworks
- **มี Controller** — รับ request จาก UI
- **มี Presenter** — แปลง output เป็น response
- **มี Gateway** — implement interface สำหรับ infrastructure

7.4.2 องค์ประกอบ

- **Controller** — รับ input จาก UI, เรียก Use Case
- **Presenter** — แปลง Use Case output เป็น view model

- **Gateway** — implement repository interface
- **Mapper** — แปลง Entity ↔ DTO/Database

7.4.3 ตัวอย่าง Controller

```
@RestController
@RequestMapping("/orders")
public class OrderController {
    private final PlaceOrderUseCase placeOrder;

    public OrderController(PlaceOrderUseCase placeOrder) {
        this.placeOrder = placeOrder;
    }

    @PostMapping
    public ResponseEntity<OrderResponse> create(@RequestBody OrderRequest req) {
        PlaceOrderCommand cmd = new PlaceOrderCommand(
            new CustomerId(req.customerId()),
            req.items().stream()
                .map(i -> new OrderItemInput(
                    new ProductId(i.productId()), i.quantity()))
                .toList()
        );

        PlaceOrderResult result = placeOrder.execute(cmd);

        return ResponseEntity.ok(new OrderResponse(
            result.orderId().value(),
            result.total().amount(),
            result.total().currency().getCode()
        ));
    }
}
```

7.4.4 ตัวอย่าง Presenter

```
public class OrderPresenter {
    public OrderViewModel present(PlaceOrderResult result) {
        return new OrderViewModel(
            result.orderId().value(),
            formatMoney(result.total())
        );
    }

    private String formatMoney(Money money) {
        return money.amount() + " " + money.currency().getCode();
```

```
    }
}
```

7.4.5 ตัวอย่าง Gateway (Repository Implementation)

```
@Repository
public class JpaOrderRepository implements OrderRepository {
    private final JpaOrderJpaRepository jpa;

    public JpaOrderRepository(JpaOrderJpaRepository jpa) {
        this.jpa = jpa;
    }

    @Override
    public Optional<Order> findById(OrderId id) {
        return jpa.findById(id.value())
            .map(this::toDomain);
    }

    @Override
    public void save(Order order) {
        jpa.save(OrderJpaEntity.fromDomain(order));
    }

    private Order toDomain(OrderJpaEntity entity) {
        // mapping
    }
}
```

7.4.6 การแปลงข้อมูล

- **Request → Command:** Controller แปลง
- **Result → Response:** Presenter แปลง
- **Entity ↔ JPA Entity:** Mapper แปลง
- **หลีกเลี่ยง:** ให้ Use Case รู้จัก DTO ของ UI

7.4.7 ข้อผิดพลาดที่พบบ่อย

- Controller ที่มี business logic
- Presenter ที่มี business logic
- Gateway ที่มี business logic
- Adapter ที่ผูกกับ Use Case มากเกินไป

7.5 ข้อดีของบท

- ทำให้เห็นบทบาทของ Interface Adapters
- เตรียมพื้นฐานสำหรับ Frameworks & Drivers
- สอนการออกแบบ Adapters ที่ดี

7.6 ข้อเสีย/ข้อจำกัด

- ต้องอาศัยความเข้าใจ Use Cases
- ต้องฝึกฝน

7.7 ข้อควรระวัง

- อย่าให้ Adapter มี business logic
- อย่าให้ Adapter ผูกกับ Use Case มากเกินไป
- อย่าละเลยการแปลงข้อมูล

7.8 ตัวอย่าง

```
Request (JSON)
    → OrderController
    → PlaceOrderCommand
    → PlaceOrderUseCase
    → Order (Entity)
    → OrderRepository (interface)
    → JpaOrderRepository (Gateway)
    → Database
```

7.9 การประยุกต์ใช้

- ใช้ออกแบบ Adapters ในระบบ
- ใช้ใน code review
- ใช้ใน architecture review

7.10 สรุป/ผลลัพธ์

ผู้อ่านออกแบบ Interface Adapters ได้ และแปลงข้อมูลระหว่างชั้น

7.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 8
- ไปสู่เล่ม 1 (Adapter pattern)

บทที่ 8: ชั้น Frameworks & Drivers

8.1 เรื่อง

ชั้น Frameworks & Drivers — เทคโนโลยีที่อยู่ชั้นนอกสุด

8.2 วัตถุประสงค์

- อธิบายบทบาทของ Frameworks & Drivers
- ทำให้เห็นว่าเทคโนโลยีควรอยู่ชั้นนอก
- เตรียมพื้นฐานสำหรับส่วนที่ 3

8.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- จัดเทคโนโลยีให้อยู่ชั้นนอก
- หลีกเลี่ยงการผูก business logic กับเทคโนโลยี
- เลือกเทคโนโลยีได้อย่างเหมาะสม

8.4 หัวข้อหลัก

8.4.1 Frameworks & Drivers คืออะไร

ชั้นนอกสุดที่ประกอบด้วย:

- **Database** — PostgreSQL, MySQL, MongoDB
- **Web Framework** — Spring, Express, Django
- **UI Framework** — React, Vue, Angular
- **External Services** — Payment Gateway, Email Service
- **Infrastructure** — Message Queue, Cache

8.4.2 บทบาท

- เป็น **เครื่องมือ** ไม่ใช่ข้อจำกัด
- **ไม่ควรมี business logic**
- **ควรเปลี่ยนได้** โดยไม่กระทบชั้นใน
- **เป็นรายละเอียด** ไม่ใช่สาระสำคัญ

8.4.3 ตัวอย่าง Frameworks & Drivers

```
// Spring Boot Application
@SpringBootApplication
public class Application {
    public static void main(String[] args) {
```

```
        SpringApplication.run(Application.class, args);
    }
}

// JPA Entity — อยู่ใน infrastructure
@Entity
@Table(name = "orders")
public class OrderJpaEntity {
    @Id
    private String id;

    @Column
    private String customerId;

    @Column
    private String status;

    // แปลงไป/กลับ
    public Order toDomain() { ... }
    public static OrderJpaEntity fromDomain(Order order) { ... }
}

// Database Config
@Configuration
public class DatabaseConfig {
    @Bean
    public DataSource dataSource() {
        return DataSourceBuilder.create()
            .url("jdbc:postgresql://localhost:5432/mydb")
            .build();
    }
}
```

8.4.4 การแยก Frameworks & Drivers

- **Business logic** อยู่ใน Entities และ Use Cases
- **Infrastructure** อยู่ในชั้นนอก
- **ใช้ interface** ในการสื่อสาร
- **Dependency** ชี้เข้าหาศูนย์กลาง

8.4.5 การเปลี่ยนเทคโนโลยี

- เปลี่ยน database: แก้ที่ Gateway
- เปลี่ยน web framework: แก้ที่ Controller
- เปลี่ยน UI: แก้ที่ Presenter
- **ไม่กระทบ** Entities และ Use Cases

8.4.6 ข้อผิดพลาดที่พบบ่อย

- Business logic ใน framework class
- Entity ผูกกับ JPA annotation
- Use Case เรียก database โดยตรง
- Framework กำหนดโครงสร้างทั้งระบบ

8.5 ข้อดีของบท

- ทำให้เห็นบทบาทของ Frameworks & Drivers
- เตรียมพื้นฐานสำหรับส่วนที่ 3
- สอนการแยกเทคโนโลยี

8.6 ข้อเสีย/ข้อจำกัด

- ต้องอาศัยความเข้าใจ Interface Adapters
- ต้องฝึกฝน

8.7 ข้อควรระวัง

- อย่าให้ framework กำหนดโครงสร้าง
- อย่าให้ business logic อยู่ใน framework class
- อย่าละเลยการแยกชั้น

8.8 ตัวอย่าง

ก่อน:

```
@Entity
public class Order {
    @Id private Long id;
    private String status;

    // Business logic ใน entity ที่ผูกกับ JPA
    public void confirm() { ... }
}
```

หลัง:

```
// Domain Entity – ไม่ผูกกับ JPA
public class Order {
    private final OrderId id;
    private OrderStatus status;
```

```
    public void confirm() { ... }
}

// JPA Entity — อยู่ใน infrastructure
@Entity
@Table(name = "orders")
public class OrderJpaEntity {
    @Id private String id;
    private String status;

    public Order toDomain() { ... }
    public static OrderJpaEntity fromDomain(Order order) { ... }
}
```

8.9 การประยุกต์ใช้

- ใช้จัดเทคโนโลยีให้อยู่ชั้นนอก
- ใช้ใน code review
- ใช้ใน architecture review

8.10 สรุป/ผลลัพธ์

ผู้อ่านจัดเทคโนโลยีให้อยู่ชั้นนอกได้ และหลีกเลี่ยงการผูก business logic

8.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่ส่วนที่ 3
- ไปสู่เล่ม 1 (Frameworks & Drivers)

ส่วนที่ 3: หลักการและแนวปฏิบัติ

บทที่ 9: การแยก Business Logic ออกจาก Database

9.1 เรื่อง

การแยก Business Logic ออกจาก Database ตามหลัก Clean Architecture

9.2 วัตถุประสงค์

- ทำให้เห็นความสำคัญของการแยก

- สอนวิธีการแยก
- เตรียมพื้นฐานสำหรับการเปลี่ยน database

9.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- แยก business logic จาก database ได้
- ออกแบบ repository interface
- เปลี่ยน database ได้โดยกระทบน้อย

9.4 หัวข้อหลัก

9.4.1 ปัญหาของการผูก Business Logic กับ Database

- ทดสอบต้องใช้ database จริง
- เปลี่ยน database ยาก
- Business logic กระจัดกระจาย
- Performance tuning กระทบ business logic

9.4.2 หลักการแยก

- **Repository interface** อยู่ที่ Use Case layer
- **Repository implementation** อยู่ที่ Infrastructure
- **Entity** ไม่มี annotation ของ ORM
- **Use Case** เรียก repository interface

9.4.3 ตัวอย่าง Repository Interface

```
// อยู่ใน Use Case layer
public interface OrderRepository {
    Optional<Order> findById(OrderId id);
    void save(Order order);
    void delete(OrderId id);
    List<Order> findByCustomer(CustomerId customerId);
}
```

9.4.4 ตัวอย่าง Repository Implementation

```
// อยู่ใน Infrastructure layer
@Repository
public class JpaOrderRepository implements OrderRepository {
    private final JpaOrderJpaRepository jpa;

    @Override
```

```
    public Optional<Order> findById(OrderId id) {
        return jpa.findById(id.value())
            .map(this::toDomain);
    }

    @Override
    public void save(Order order) {
        jpa.save(OrderJpaEntity.fromDomain(order));
    }

    private Order toDomain(OrderJpaEntity entity) {
        // mapping
    }
}
```

9.4.5 Entity ที่ไม่ผูกกับ Database

```
// Domain Entity
public class Order {
    private final OrderId id;
    private final List<OrderItem> items;
    private OrderStatus status;

    public void confirm() {
        if (items.isEmpty()) {
            throw new IllegalStateException("Cannot confirm empty order");
        }
        this.status = OrderStatus.CONFIRMED;
    }
}

// JPA Entity — อยู่ใน infrastructure
@Entity
@Table(name = "orders")
public class OrderJpaEntity {
    @Id private String id;

    @OneToMany(cascade = CascadeType.ALL)
    private List<OrderItemJpaEntity> items;

    private String status;

    public Order toDomain() {
        return new Order(
            new OrderId(id),
            items.stream().map(OrderItemJpaEntity::toDomain).toList(),
```

```
            OrderStatus.valueOf(status)
        );
    }

    public static OrderJpaEntity fromDomain(Order order) {
        // mapping
    }
}
```

9.4.6 การทดสอบ

- Entity ทดสอบโดยตรง
- Use Case ทดสอบด้วย mock repository
- Repository ทดสอบด้วย integration test
- ไม่ต้องใช้ database สำหรับ business logic

9.4.7 การเปลี่ยน Database

- แก้ที่ Repository implementation
- ไม่กระทบ Entity และ Use Case
- อาจต้องแก้ mapping
- ใช้ migration tools

9.5 ข้อดีของบท

- ทำให้เห็นความสำคัญของการแยก
- สอนวิธีการแยก
- เตรียมพื้นฐานสำหรับการเปลี่ยน database

9.6 ข้อเสีย/ข้อจำกัด

- ต้องใช้ความพยายามในการแยก
- อาจต้องเขียน mapping code

9.7 ข้อควรระวัง

- อย่าให้ Entity ผูกกับ JPA
- อย่าให้ Use Case เรียก database โดยตรง
- อย่าละเลย mapping

9.8 ตัวอย่าง

ก่อน:

```java
@Entity
public class Order {
    @Id private Long id;

    // Business logic ใน entity ที่ผูกกับ JPA
    public Money total() { ... }
}

// Use Case เรียก repository ที่ผูกกับ JPA
public class PlaceOrderUseCase {
    @Autowired
    private OrderJpaRepository jpa;  // ผูกกับ JPA

    public void execute(...) {
        jpa.save(...);
    }
}
```

หลัง:

```java
// Domain Entity
public class Order {
    private final OrderId id;
    public Money total() { ... }
}

// Use Case ใช้ interface
public class PlaceOrderUseCase {
    private final OrderRepository orders;  // interface

    public void execute(...) {
        orders.save(...);
    }
}

// Infrastructure
@Repository
public class JpaOrderRepository implements OrderRepository {
    // implementation
}
```

9.9 การประยุกต์ใช้

- ใช้ออกแบบ repository ในระบบ
- ใช้ใน code review

- ใช้ใน architecture review

9.10 สรุป/ผลลัพธ์

ผู้อ่านแยก business logic จาก database ได้

9.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 10
- ไปสู่เล่ม 3 (Repository)

บทที่ 10: การแยก Business Logic ออกจาก Framework

10.1 เรื่อง

การแยก Business Logic ออกจาก Framework ตามหลัก Clean Architecture

10.2 วัตถุประสงค์

- ทำให้เห็นความสำคัญของการแยก
- สอนวิธีการแยก
- เตรียมพื้นฐานสำหรับการเปลี่ยน framework

10.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- แยก business logic จาก framework ได้
- ใช้ framework เป็นเครื่องมือ
- เปลี่ยน framework ได้โดยกระทบน้อย

10.4 หัวข้อหลัก

10.4.1 ปัญหาของการผูก Business Logic กับ Framework

- ทดสอบต้องใช้ framework
- เปลี่ยน framework ยาก
- Business logic ไม่ portable
- Framework กำหนดโครงสร้าง

10.4.2 หลักการแยก

- **Entities และ Use Cases** ไม่มี framework annotation
- **Framework** อยู่ที่ Infrastructure

- **ใช้ interface** ในการสื่อสาร
- **Dependency** ชี้เข้าหาศูนย์กลาง

10.4.3 ตัวอย่าง Entity ที่ไม่ผูกกับ Framework

```
// Domain Entity — ไม่มี framework annotation
public class Order {
    private final OrderId id;
    private final List<OrderItem> items;
    private OrderStatus status;

    public void confirm() {
        if (items.isEmpty()) {
            throw new IllegalStateException("Cannot confirm empty order");
        }
        this.status = OrderStatus.CONFIRMED;
    }
}
```

10.4.4 ตัวอย่าง Use Case ที่ไม่ผูกกับ Framework

```
// Use Case — ไม่มี @Service, @Component
public class PlaceOrderUseCase {
    private final OrderRepository orders;
    private final EventPublisher events;

    public PlaceOrderUseCase(OrderRepository orders, EventPublisher events) {
        this.orders = orders;
        this.events = events;
    }

    public OrderId execute(PlaceOrderCommand cmd) {
        Order order = new Order(...);
        orders.save(order);
        events.publish(new OrderPlaced(order.id()));
        return order.id();
    }
}
```

10.4.5 การใช้ Framework เป็นเครื่องมือ

```
// Infrastructure — ใช้ Spring
@Configuration
public class UseCaseConfig {
    @Bean
```

```
    public PlaceOrderUseCase placeOrderUseCase(
            OrderRepository orders,
            EventPublisher events) {
        return new PlaceOrderUseCase(orders, events);
    }
}
```

10.4.6 การทดสอบ

- Entity ทดสอบโดยตรง
- Use Case ทดสอบด้วย mock
- ไม่ต้องใช้ framework
- เร็ว, ครอบคลุม

10.4.7 การเปลี่ยน Framework

- แก้ที่ Infrastructure
- ไม่กระทบ Entity และ Use Case
- อาจต้องแก้ configuration
- ใช้ adapter pattern

10.5 ข้อดีของบท

- ทำให้เห็นความสำคัญของการแยก
- สอนวิธีการแยก
- เตรียมพื้นฐานสำหรับการเปลี่ยน framework

10.6 ข้อเสีย/ข้อจำกัด

- ต้องใช้ความพยายามในการแยก
- อาจต้องเขียน configuration เพิ่ม

10.7 ข้อควรระวัง

- อย่าให้ Entity ผูกกับ framework
- อย่าให้ Use Case ผูกกับ framework
- อย่าละเลยการ config

10.8 ตัวอย่าง

ก่อน:

```
@Service
public class OrderService {
```

```
    @Autowired
    private OrderRepository repo;

    @Transactional
    public void placeOrder(...) {
        // business logic
    }
}
```

หลัง:

```
// Use Case — ไม่มี annotation
public class PlaceOrderUseCase {
    private final OrderRepository orders;

    public PlaceOrderUseCase(OrderRepository orders) {
        this.orders = orders;
    }

    public void execute(...) {
        // orchestration only
    }
}

// Infrastructure — config
@Configuration
public class Config {
    @Bean
    public PlaceOrderUseCase placeOrderUseCase(OrderRepository orders) {
        return new PlaceOrderUseCase(orders);
    }
}
```

10.9 การประยุกต์ใช้

- ใช้ออกแบบ Use Cases ในระบบ
- ใช้ใน code review
- ใช้ใน architecture review

10.10 สรุป/ผลลัพธ์

ผู้อ่านแยก business logic จาก framework ได้

10.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 11
- ไปสู่เล่ม 1 (Framework independence)

บทที่ 11: การแยก Business Logic ออกจาก UI

11.1 เรื่อง

การแยก Business Logic ออกจาก UI ตามหลัก Clean Architecture

11.2 วัตถุประสงค์

- ทำให้เห็นความสำคัญของการแยก
- สอนวิธีการแยก
- เตรียมพื้นฐานสำหรับการเปลี่ยน UI

11.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- แยก business logic จาก UI ได้
- ออกแบบ controller/presenter
- เปลี่ยน UI ได้โดยกระทบน้อย

11.4 หัวข้อหลัก

11.4.1 ปัญหาของการผูก Business Logic กับ UI

- ทดสอบต้องใช้ UI
- เปลี่ยน UI ยาก
- Business logic ไม่ reusable
- UI framework กำหนดโครงสร้าง

11.4.2 หลักการแยก

- **Business logic** อยู่ใน Entities และ Use Cases
- **UI** อยู่ใน Interface Adapters และ Frameworks
- **Controller** รับ request, เรียก Use Case
- **Presenter** แปลง output เป็น view model

11.4.3 ตัวอย่าง Controller

```
// Interface Adapter — ไม่มี business logic
@RestController
public class OrderController {
    private final PlaceOrderUseCase placeOrder;

    @PostMapping("/orders")
    public ResponseEntity<OrderResponse> create(@RequestBody OrderRequest req) {
        PlaceOrderCommand cmd = toCommand(req);
        PlaceOrderResult result = placeOrder.execute(cmd);
        return ResponseEntity.ok(toResponse(result));
    }

    private PlaceOrderCommand toCommand(OrderRequest req) {
        // แปลง request เป็น command
    }

    private OrderResponse toResponse(PlaceOrderResult result) {
        // แปลง result เป็น response
    }
}
```

11.4.4 ตัวอย่าง Presenter

```
// Interface Adapter — แปลง output
public class OrderPresenter {
    public OrderViewModel present(PlaceOrderResult result) {
        return new OrderViewModel(
            result.orderId().value(),
            formatMoney(result.total())
        );
    }
}
```

11.4.5 การทดสอบ

- Entity ทดสอบโดยตรง
- Use Case ทดสอบด้วย mock
- Controller ทดสอบด้วย mock Use Case
- ไม่ต้องใช้ UI จริง

11.4.6 การเปลี่ยน UI

- แก้ที่ Controller/Presenter

- ไม่กระทบ Entity และ Use Case
- อาจต้องแก้ view model
- ใช้ adapter pattern

11.5 ข้อดีของบท

- ทำให้เห็นความสำคัญของการแยก
- สอนวิธีการแยก
- เตรียมพื้นฐานสำหรับการเปลี่ยน UI

11.6 ข้อเสีย/ข้อจำกัด

- ต้องใช้ความพยายามในการแยก
- อาจต้องเขียน DTO เพิ่ม

11.7 ข้อควรระวัง

- อย่าให้ Controller มี business logic
- อย่าให้ Use Case รู้จัก UI
- อย่าละเลยการแปลงข้อมูล

11.8 ตัวอย่าง

ก่อน:

```
@RestController
public class OrderController {
    @Autowired
    private JdbcTemplate jdbc;

    @PostMapping("/orders")
    public ResponseEntity<?> create(@RequestBody OrderRequest req) {
        // Business logic ใน controller
        if (req.getItems().isEmpty()) {
            return ResponseEntity.badRequest().build();
        }
        // ...
    }
}
```

หลัง:

```
@RestController
public class OrderController {
```

```
    private final PlaceOrderUseCase placeOrder;

    @PostMapping("/orders")
    public ResponseEntity<OrderResponse> create(@RequestBody OrderRequest req) {
        PlaceOrderCommand cmd = toCommand(req);
        PlaceOrderResult result = placeOrder.execute(cmd);
        return ResponseEntity.ok(toResponse(result));
    }
}
```

11.9 การประยุกต์ใช้

- ใช้ออกแบบ Controller/Presenter
- ใช้ใน code review
- ใช้ใน architecture review

11.10 สรุป/ผลลัพธ์

ผู้อ่านแยก business logic จาก UI ได้

11.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 12
- ไปสู่เล่ม 1 (UI independence)

บทที่ 12: Dependency Injection และ Inversion of Control

12.1 เรื่อง

Dependency Injection (DI) และ Inversion of Control (IoC) ในบริบทของ Clean Architecture

12.2 วัตถุประสงค์

- อธิบาย DI และ IoC
- ทำให้เห็นบทบาทใน Clean Architecture
- เตรียมพื้นฐานสำหรับการทดสอบ

12.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- ใช้ DI ใน Clean Architecture
- ออกแบบ IoC ที่เหมาะสม

- ทดสอบได้ง่าย

12.4 หัวข้อหลัก

12.4.1 Dependency Injection คืออะไร

DI คือเทคนิคที่:

- Object ไม่สร้าง dependency เอง
- Dependency ถูก inject จากภายนอก
- ลด coupling
- ทำให้ทดสอบง่าย

12.4.2 รูปแบบของ DI

- **Constructor Injection** — ผ่าน constructor (แนะนำ)
- **Setter Injection** — ผ่าน setter
- **Field Injection** — ผ่าน field (ไม่แนะนำ)

12.4.3 ตัวอย่าง Constructor Injection

```
public class PlaceOrderUseCase {
    private final OrderRepository orders;
    private final EventPublisher events;

    // Constructor Injection
    public PlaceOrderUseCase(OrderRepository orders, EventPublisher events) {
        this.orders = orders;
        this.events = events;
    }

    public OrderId execute(PlaceOrderCommand cmd) {
        Order order = new Order(...);
        orders.save(order);
        events.publish(new OrderPlaced(order.id()));
        return order.id();
    }
}
```

12.4.4 Inversion of Control

IoC คือหลักการที่:

- การควบคุมถูก invert
- Object ไม่ควบคุม dependency
- Framework/container ควบคุม

- Dependency ชี้เข้าหา abstraction

12.4.5 DI Container

- Spring, Guice, Dagger
- จัดการ lifecycle ของ object
- inject dependency อัตโนมัติ
- config ที่ Infrastructure

12.4.6 การทดสอบด้วย DI

```
@Test
void placeOrder_savesOrder() {
    // Arrange
    OrderRepository mockRepo = mock(OrderRepository.class);
    EventPublisher mockEvents = mock(EventPublisher.class);
    PlaceOrderUseCase useCase = new PlaceOrderUseCase(mockRepo, mockEvents);

    // Act
    useCase.execute(new PlaceOrderCommand(...));

    // Assert
    verify(mockRepo).save(any(Order.class));
}
```

12.4.7 ข้อผิดพลาดที่พบบ่อย

- Field injection
- Service locator
- DI container ใน domain
- ซับซ้อนเกินจำเป็น

12.5 ข้อดีของบท

- ทำให้เห็นบทบาทของ DI
- เตรียมพื้นฐานสำหรับการทดสอบ
- สอนการออกแบบที่ดี

12.6 ข้อเสีย/ข้อจำกัด

- อาจเพิ่ม complexity
- ต้องใช้ความเข้าใจ

12.7 ข้อควรระวัง

- อย่าใช้ field injection
- อย่าใช้ service locator
- อย่าให้ DI container อยู่ใน domain
- อย่าซับซ้อนเกินจำเป็น

12.8 ตัวอย่าง

```
// Configuration — Infrastructure
@Configuration
public class AppConfig {
    @Bean
    public OrderRepository orderRepository(JpaOrderJpaRepository jpa) {
        return new JpaOrderRepository(jpa);
    }

    @Bean
    public EventPublisher eventPublisher(ApplicationEventPublisher publisher) {
        return new SpringEventPublisher(publisher);
    }

    @Bean
    public PlaceOrderUseCase placeOrderUseCase(
            OrderRepository orders,
            EventPublisher events) {
        return new PlaceOrderUseCase(orders, events);
    }
}
```

12.9 การประยุกต์ใช้

- ใช้ออกแบบ DI ในระบบ
- ใช้ใน code review
- ใช้ในการทดสอบ

12.10 สรุป/ผลลัพธ์

ผู้อ่านใช้ DI ใน Clean Architecture ได้ และทดสอบได้ง่าย

12.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 13
- ไปสู่เล่ม 1 (Dependency Injection)

ส่วนที่ 4: การทดสอบและการเปลี่ยนเทคโนโลยี

บทที่ 13: วิธีทดสอบ Clean Architecture

13.1 เรื่อง

การทดสอบแต่ละชั้นใน Clean Architecture

13.2 วัตถุประสงค์

- อธิบายการทดสอบแต่ละชั้น
- ทำให้เห็นว่า Clean Architecture ทดสอบง่าย
- เตรียมพื้นฐานสำหรับ test strategy

13.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- ออกแบบ test strategy ได้
- ทดสอบแต่ละชั้นได้
- ใช้ test pyramid

13.4 หัวข้อหลัก

13.4.1 Test Pyramid ใน Clean Architecture

```
        ┌──────────────┐
        │   E2E Test   │  น้อย
        ├──────────────┤
        │ Integration  │  กลาง
        ├──────────────┤
        │  Unit Test   │  มาก
        └──────────────┘
```

13.4.2 การทดสอบ Entities

- Unit test โดยตรง
- ไม่ต้องใช้ mock
- เร็ว, ครอบคลุม

- ทดสอบ business rules

```
@Test
void confirm_emptyOrder_throwsException() {
    Order order = new Order(
        OrderId.generate(),
        CustomerId.generate(),
        List.of()
    );

    assertThrows(IllegalStateException.class, order::confirm);
}
```

13.4.3 การทดสอบ Use Cases

- Unit test ด้วย mock repository
- ไม่ต้องใช้ database
- ทดสอบ orchestration

```
@Test
void placeOrder_savesOrder() {
    OrderRepository mockRepo = mock(OrderRepository.class);
    EventPublisher mockEvents = mock(EventPublisher.class);
    PlaceOrderUseCase useCase = new PlaceOrderUseCase(mockRepo, mockEvents);

    useCase.execute(new PlaceOrderCommand(...));

    verify(mockRepo).save(any(Order.class));
}
```

13.4.4 การทดสอบ Interface Adapters

- Unit test controller ด้วย mock Use Case
- Unit test presenter
- Integration test gateway

```
@Test
void createOrder_returnsOk() {
    PlaceOrderUseCase mockUseCase = mock(PlaceOrderUseCase.class);
    OrderController controller = new OrderController(mockUseCase);

    when(mockUseCase.execute(any())).thenReturn(new PlaceOrderResult(...));

    ResponseEntity<OrderResponse> response = controller.create(new OrderRequest(...));
```

```
    assertEquals(HttpStatus.OK, response.getStatusCode());
}
```

13.4.5 การทดสอบ Frameworks & Drivers

- Integration test
- ใช้ database จริง (หรือ testcontainers)
- ทดสอบ repository implementation

```
@DataJpaTest
class JpaOrderRepositoryTest {
    @Autowired
    private JpaOrderRepository repo;

    @Test
    void save_and_findById() {
        Order order = new Order(...);
        repo.save(order);

        Optional<Order> found = repo.findById(order.id());

        assertTrue(found.isPresent());
    }
}
```

13.4.6 Test Strategy

- **Entities:** unit test ครอบคลุม business rules
- **Use Cases:** unit test ครอบคลุม orchestration
- **Adapters:** unit test + integration test
- **Frameworks:** integration test
- **E2E:** เฉพาะ critical flows

13.4.7 ข้อผิดพลาดที่พบบ่อย

- ทดสอบ business logic ผ่าน UI
- ทดสอบ Use Case ด้วย database จริง
- ละเลย unit test
- E2E test มากเกินไป

13.5 ข้อดีของบท

- ทำให้เห็นว่า Clean Architecture ทดสอบง่าย

- เตรียมพื้นฐานสำหรับ test strategy
- สอนการทดสอบแต่ละชั้น

13.6 ข้อเสีย/ข้อจำกัด

- ต้องใช้ความเข้าใจ test pyramid
- ต้องฝึกฝน

13.7 ข้อควรระวัง

- อย่าละเลย unit test
- อย่า E2E test มากเกินไป
- อย่าทดสอบ business logic ผ่าน UI

13.8 ตัวอย่าง

Test strategy สำหรับระบบ e-commerce:

- **Entities:** unit test สำหรับ Order, Money
- **Use Cases:** unit test สำหรับ PlaceOrderUseCase
- **Adapters:** unit test สำหรับ Controller, Presenter
- **Frameworks:** integration test สำหรับ JpaOrderRepository
- **E2E:** ทดสอบ critical flow (place order → payment → shipping)

13.9 การประยุกต์ใช้

- ใช้ Test Strategy Template (ภาคผนวก D)
- ใช้ใน code review
- ใช้ใน CI/CD

13.10 สรุป/ผลลัพธ์

ผู้ออกแบบ test strategy ได้ และทดสอบแต่ละชั้นได้

13.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 14
- ไปสู่เล่ม 1 (Testability)

บทที่ 14: วิธีเปลี่ยนเทคโนโลยีโดยกระทบน้อยที่สุด

14.1 เรื่อง

การเปลี่ยนเทคโนโลยีใน Clean Architecture โดยกระทบน้อยที่สุด

14.2 วัตถุประสงค์

- ทำให้เห็นว่า Clean Architecture ช่วยให้เปลี่ยน technoloy ได้ง่าย
- สอนวิธีการเปลี่ยน
- เตรียมพื้นฐานสำหรับการ evolve ระบบ

14.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- เปลี่ยน database ได้โดยกระทบน้อย
- เปลี่ยน framework ได้โดยกระทบน้อย
- เปลี่ยน UI ได้โดยกระทบน้อย

14.4 หัวข้อหลัก

14.4.1 การเปลี่ยน Database

ขั้นตอน:

1. สร้าง implementation ใหม่ของ repository
2. เขียน migration
3. ทดสอบ
4. เปลี่ยน config
5. ลบ implementation เก่า

กระทบ:

- Infrastructure layer
- Config
- **ไม่กระทบ:** Entities, Use Cases

14.4.2 การเปลี่ยน Web Framework

ขั้นตอน:

1. สร้าง controller ใหม่
2. แปลง request/response
3. ทดสอบ
4. เปลี่ยน routing

5. ลบ controller เก่า

กระทบ:

- Interface Adapters
- Infrastructure
- **ไม่กระทบ:** Entities, Use Cases

14.4.3 การเปลี่ยน UI

ขั้นตอน:

1. สร้าง presenter ใหม่
2. สร้าง view model ใหม่
3. ทดสอบ
4. เปลี่ยน frontend
5. ลบ presenter เก่า

กระทบ:

- Interface Adapters
- Frameworks
- **ไม่กระทบ:** Entities, Use Cases

14.4.4 ตัวอย่าง: เปลี่ยนจาก MySQL เป็น PostgreSQL

```
// ก่อน – MySQL
@Repository
public class MySqlOrderRepository implements OrderRepository {
    private final JdbcTemplate jdbc;
    // MySQL-specific
}

// หลัง – PostgreSQL
@Repository
public class PostgresOrderRepository implements OrderRepository {
    private final JdbcTemplate jdbc;
    // PostgreSQL-specific
}

// Use Case ไม่เปลี่ยน
public class PlaceOrderUseCase {
    private final OrderRepository orders;  // interface เดิม
}
```

14.4.5 ตัวอย่าง: เปลี่ยนจาก Spring MVC เป็น Quarkus

```
// ก่อน – Spring
@RestController
public class OrderController {
    @GetMapping("/orders/{id}")
    public OrderResponse get(@PathVariable String id) { ... }
}

// หลัง – Quarkus
@Path("/orders")
public class OrderResource {
    @GET
    @Path("/{id}")
    public OrderResponse get(@PathParam("id") String id) { ... }
}

// Use Case ไม่เปลี่ยน
```

14.4.6 ข้อผิดพลาดที่พบบ่อย

- Business logic ใน infrastructure
- Entity ผูกกับ framework
- Use Case เรียก database โดยตรง
- ไม่มี interface

14.5 ข้อดีของบท

- ทำให้เห็นว่า Clean Architecture ช่วยให้เปลี่ยน technology ได้ง่าย
- เตรียมพื้นฐานสำหรับการ evolve
- สอนวิธีการเปลี่ยน

14.6 ข้อเสีย/ข้อจำกัด

- ต้องใช้ความพยายามในการแยก
- ต้องมี interface ที่ดี

14.7 ข้อควรระวัง

- อย่าให้ business logic อยู่ใน infrastructure
- อย่าให้ Entity ผูกกับ framework
- อย่าละเลย interface

14.8 ตัวอย่าง

การเปลี่ยน Database:

- แก้ที่ `JpaOrderRepository`
- ไม่แก้ `Order`, `PlaceOrderUseCase`
- ใช้ migration tools

การเปลี่ยน Framework:

- แก้ที่ `OrderController`
- ไม่แก้ `PlaceOrderUseCase`
- ใช้ adapter

14.9 การประยุกต์ใช้

- ใช้ Technology Migration Checklist (ภาคผนวก E)
- ใช้ใน POC
- ใช้ใน architecture review

14.10 สรุป/ผลลัพธ์

ผู้อ่านเปลี่ยนเทคโนโลยีได้โดยกระทบน้อย

14.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 15
- ไปสู่เล่ม 1 (Technology independence)

ส่วนที่ 5: การนำไปใช้และกรณีศึกษา

บทที่ 15: ข้อดี/ข้อเสียของ Clean Architecture

15.1 เรื่อง

การวิเคราะห์ข้อดี/ข้อเสียของ Clean Architecture อย่างละเอียด

15.2 วัตถุประสงค์

- ทำให้เห็น trade-offs อย่างครบถ้วน
- ช่วยตัดสินใจได้อย่างมีข้อมูล

- ลดการเลือกตามกระแส

15.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- วิเคราะห์ข้อดี/ข้อเสียได้
- ตัดสินใจได้อย่างมีเหตุผล
- เตรียมความพร้อมสำหรับความท้าทาย

15.4 หัวข้อหลัก

15.4.1 ข้อดีของ Clean Architecture

- **Testability** — business logic ทดสอบง่าย
- **Independence** — ไม่ผูกกับ framework, database, UI
- **Maintainability** — แยกชั้นชัดเจน
- **Flexibility** — เปลี่ยนเทคโนโลยีง่าย
- **Business logic ชัดเจน** — อยู่ใน Entities และ Use Cases
- **Parallel development** — ทีมทำงานแยกชั้นได้

15.4.2 ข้อเสียของ Clean Architecture

- **Complexity** — หลายชั้น เพิ่ม complexity
- **Boilerplate** — ต้องเขียน interface, DTO, mapper
- **Learning curve** — ต้องเข้าใจหลักการ
- **Over-engineering** — อาจเกินจำเป็นสำหรับระบบเล็ก
- **Performance** — mapping หลายชั้นอาจกระทบ performance
- **Initial cost** — ใช้เวลาในช่วงเริ่มต้น

15.4.3 เมื่อใดควรใช้ Clean Architecture

เหมาะกับ:

- ระบบที่มี business logic ซับซ้อน
- ระบบที่ต้องทดสอบ business logic
- ระบบที่ต้องเปลี่ยนเทคโนโลยีบ่อย
- ระบบระยะยาว
- ทีมที่มีประสบการณ์

ไม่เหมาะกับ:

- ระบบ CRUD ง่าย ๆ
- Prototype
- ระบบขนาดเล็กมาก

- ทีมที่ยังไม่มีประสบการณ์
- โครงการที่ต้องการ speed มากกว่า correctness

15.4.4 ทางเลือกกลาง

- **Hexagonal** — เบากว่า Clean
- **Onion** — คล้าย Clean แต่ยืดหยุ่นกว่า
- **Layered** — ง่ายกว่า แต่มีข้อจำกัด
- **Modular Monolith** — แบ่ง module โดยไม่ต้องมี 4 ชั้น

15.4.5 การตัดสินใจ

การตัดสินใจควรอิง:

- ความซับซ้อนของ business logic
- ขนาดระบบ
- ขนาดทีม
- ประสบการณ์ทีม
- ระยะเวลาโครงการ
- ความต้องการทดสอบ

15.5 ข้อดีของบท

- ทำให้เห็น trade-offs อย่างครบถ้วน
- ช่วยตัดสินใจได้อย่างมีข้อมูล
- เตรียมความพร้อมสำหรับความท้าทาย

15.6 ข้อเสีย/ข้อจำกัด

- การเปรียบเทียบเป็น contextual
- ตัวอย่างอาจไม่ครอบคลุมทุกกรณี

15.7 ข้อควรระวัง

- อย่าเลือกตามกระแส
- อย่าละเลยความพร้อมของทีม
- อย่า over-engineer

15.8 ตัวอย่าง

กรณีที่ 1: ระบบ ERP ที่มี business logic ซับซ้อน → Clean Architecture เหมาะสม

กรณีที่ 2: ระบบ CRUD ง่าย ๆ → ไม่จำเป็น

กรณีที่ 3: ระบบที่มี business logic ปานกลาง → Hexagonal หรือ Modular Monolith

15.9 การประยุกต์ใช้

- ใช้ใน workshop ตัดสินใจ
- ใช้สื่อสารกับผู้มีส่วนได้ส่วนเสีย
- ใช้ใน architecture review

15.10 สรุป/ผลลัพธ์

ผู้อ่านวิเคราะห์ข้อดี/ข้อเสียได้ และตัดสินใจได้อย่างมีเหตุผล

15.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 16
- ไปสู่เล่ม 1 (Architecture trade-offs)

บทที่ 16: ตัวอย่างการออกแบบระบบด้วย Clean Architecture

16.1 เรื่อง

กรณีศึกษาการออกแบบระบบด้วย Clean Architecture ตั้งแต่ต้นจนจบ

16.2 วัตถุประสงค์

- ประยุกต์แนวคิดทั้งหมดกับกรณีศึกษา
- ทำให้เห็นภาพรวมการออกแบบ
- เตรียมผู้อ่านสำหรับการออกแบบระบบของตนเอง

16.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- ออกแบบระบบด้วย Clean Architecture ได้
- ใช้เครื่องมือจากบทก่อน ๆ
- วิเคราะห์ trade-offs

16.4 หัวข้อหลัก

16.4.1 บริบท

ระบบ e-commerce สำหรับ SME:

- ผู้ใช้: 1,000 คน/วัน
- ทีม: 10 คน
- Business logic ปานกลาง

- ต้องการทดสอบ business logic
- ต้องการเปลี่ยน database ได้ในอนาคต

16.4.2 Entities

```java
// Order
public class Order {
    private final OrderId id;
    private final CustomerId customerId;
    private final List<OrderItem> items;
    private OrderStatus status;

    public void confirm() { ... }
    public Money total() { ... }
}

// Money
public class Money {
    private final BigDecimal amount;
    private final Currency currency;

    public Money add(Money other) { ... }
}
```

16.4.3 Use Cases

```java
// PlaceOrderUseCase
public class PlaceOrderUseCase {
    private final OrderRepository orders;
    private final EventPublisher events;

    public OrderId execute(PlaceOrderCommand cmd) {
        Order order = new Order(...);
        orders.save(order);
        events.publish(new OrderPlaced(order.id()));
        return order.id();
    }
}

// OrderRepository — interface
public interface OrderRepository {
    Optional<Order> findById(OrderId id);
    void save(Order order);
}
```

16.4.4 Interface Adapters

```java
// Controller
@RestController
public class OrderController {
    private final PlaceOrderUseCase placeOrder;

    @PostMapping("/orders")
    public ResponseEntity<OrderResponse> create(@RequestBody OrderRequest req) {
        PlaceOrderResult result = placeOrder.execute(toCommand(req));
        return ResponseEntity.ok(toResponse(result));
    }
}

// Repository Implementation
@Repository
public class JpaOrderRepository implements OrderRepository {
    // implementation
}
```

16.4.5 Frameworks & Drivers

```java
// Spring Boot
@SpringBootApplication
public class Application { ... }

// JPA Entity
@Entity
@Table(name = "orders")
public class OrderJpaEntity { ... }
```

16.4.6 โครงสร้างโค้ด

```
src/
├── domain/
│   ├── Order.java
│   ├── OrderItem.java
│   ├── Money.java
│   └── OrderId.java
├── usecase/
│   ├── PlaceOrderUseCase.java
│   ├── PlaceOrderCommand.java
│   ├── PlaceOrderResult.java
│   └── OrderRepository.java
├── adapter/
```

```
│   ├── controller/
│   │   └── OrderController.java
│   ├── presenter/
│   │   └── OrderPresenter.java
│   └── gateway/
│       └── JpaOrderRepository.java
└── infrastructure/
    ├── Application.java
    └── config/
        └── AppConfig.java
```

16.4.7 **การทดสอบ**

- **Entities:** unit test
- **Use Cases:** unit test ด้วย mock
- **Adapters:** unit test + integration test
- **E2E:** critical flow

16.4.8 **บทเรียน**

- Clean Architecture ทำให้ทดสอบง่าย
- การแยกชั้นช่วยให้เปลี่ยนเทคโนโลยีได้
- Boilerplate เป็นเรื่องที่ต้องยอมรับ
- ต้องมีวินัยในการรักษา Dependency Rule

16.5 ข้อดีของบท

- ทำให้เห็นภาพรวมการออกแบบ
- เชื่อมแนวคิดทั้งหมดเข้าด้วยกัน
- เตรียมผู้อ่านสำหรับการออกแบบระบบของตนเอง

16.6 ข้อเสีย/ข้อจำกัด

- เป็นกรณีศึกษาเดียว
- ตัวอย่างอาจไม่ครอบคลุมทุกกรณี

16.7 ข้อควรระวัง

- อย่าคัดลอกโดยไม่ปรับบริบท
- อย่าละเลย trade-offs
- อย่าละเลยการวัดผล

16.8 ตัวอย่าง

Order flow:

1. User สร้าง order ผ่าน REST API
2. OrderController แปลง request เป็น command
3. PlaceOrderUseCase orchestrate
4. Order entity ถูกสร้าง
5. OrderRepository save
6. Event ถูก publish
7. Response ถูกส่งกลับ

16.9 การประยุกต์ใช้

- ใช้เป็น template สำหรับระบบของตนเอง
- ใช้ใน workshop
- ใช้ใน architecture review

16.10 สรุป/ผลลัพธ์

ผู้อ่านออกแบบระบบด้วย Clean Architecture ได้ และเห็นภาพรวมการทำงาน

16.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 17
- ไปสู่เล่ม 3 (DDD)

บทที่ 17: การประยุกต์ใช้และสรุปเล่ม

17.1 เรื่อง

การนำเนื้อหาทั้งเล่มไปใช้ในองค์กร

17.2 วัตถุประสงค์

- สรุปสาระสำคัญของเล่ม
- แปลงความรู้เป็นแผนปฏิบัติ
- เชื่อมไปยังเล่มถัดไป

17.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- มีแผนประยุกต์ใช้ในองค์กร
- จัด workshop ตามแม่แบบในเล่ม
- รู้ว่าต้องไปอ่านเล่มใดต่อ

17.4 หัวข้อหลัก

17.4.1 Workshop: Layer Dependency Review

- ใช้ภาคผนวก A
- ตรวจ dependency ระหว่างชั้น
- ระบุการละเมิด

17.4.2 Workshop: Use Case Design

- ใช้ภาคผนวก B
- ออกแบบ Use Case
- กำหนด interface

17.4.3 Workshop: Test Strategy

- ใช้ภาคผนวก D
- ออกแบบ test strategy
- กำหนด test แต่ละชั้น

17.4.4 สรุปสาระสำคัญ

- Clean Architecture แยก business logic จากเทคโนโลยี
- Dependency Rule เป็นหัวใจ
- สี่ชั้น: Entities, Use Cases, Interface Adapters, Frameworks & Drivers
- ทดสอบง่าย
- เปลี่ยนเทคโนโลยีได้โดยกระทบน้อย
- มี trade-offs

17.4.5 เชื่อมไปเล่ม 5–7

- เล่ม 5: ความเสี่ยง
- เล่ม 6: POC
- เล่ม 7: การบริหารโครงการ

17.5 ข้อดีของบท

- ปิดเล่มด้วยการลงมือทำ
- ทำให้ความรู้แปลงเป็นแผนปฏิบัติ
- เชื่อมโยงกับเล่มถัดไปอย่างชัดเจน

17.6 ข้อเสีย/ข้อจำกัด

- ต้องปรับตามบริบทองค์กร
- ต้องมีผู้นำ workshop

17.7 ข้อควรระวัง

- อย่านำไปใช้แบบคัดลอกโดยไม่ปรับบริบท
- อย่าทำ workshop ครั้งเดียวแล้วจบ
- อย่าละเลยการติดตามผล

17.8 ตัวอย่าง

แผนนำไปใช้ใน 90 วัน:

- **เดือน 1:** Layer Dependency Review + Use Case Design
- **เดือน 2:** POC สำหรับ Use Case แรก
- **เดือน 3:** ขยายไปยัง Use Case อื่น + ตั้ง test strategy

17.9 การประยุกต์ใช้

- ใช้เป็นแผนดำเนินงานหลังอ่านจบ
- ใช้เป็น checklist ของทีม
- ใช้เป็นพื้นฐานสำหรับเล่มถัดไป

17.10 สรุป/ผลลัพธ์

ผู้อ่านมีแผนประยุกต์ใช้ในองค์กร และรู้เส้นทางการอ่านต่อ

17.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่เล่ม 5–7 ทั้งหมด

ภาคผนวก

ภาคผนวก A: Layer Dependency Checklist

- [ ] Entities ไม่ import จากชั้นอื่น
- [ ] Use Cases import เฉพาะ Entities
- [ ] Interface Adapters import Use Cases และ Entities
- [ ] Frameworks & Drivers import ได้ทุกชั้น
- [ ] ไม่มี framework annotation ใน Entities
- [ ] ไม่มี database logic ใน Use Cases
- [ ] Repository interface อยู่ที่ Use Case layer
- [ ] Repository implementation อยู่ที่ Infrastructure

- [ ] Dependency ชี้เข้าหาศูนย์กลางเสมอ

ภาคผนวก B: Use Case Template

หัวข้อ	รายละเอียด
ชื่อ Use Case	PlaceOrder
Actor	Customer
Trigger	Customer checkout
Input	PlaceOrderCommand
Output	PlaceOrderResult
Steps	1. Validate items 2. Create order 3. Save 4. Publish event
Dependencies	OrderRepository, EventPublisher
Errors	ProductNotFound, InvalidQuantity
Transaction	ใช้ @Transactional ที่ adapter

ภาคผนวก C: Interface Adapter Checklist

- [ ] Controller ไม่มี business logic
- [ ] Controller แปลง request เป็น command
- [ ] Presenter แปลง result เป็น view model
- [ ] Gateway implement repository interface
- [ ] Mapper แปลง Entity ↔ DTO/Database
- [ ] ไม่มี framework annotation ที่ไม่จำเป็น
- [ ] Adapter ไม่รู้จัก Entity โดยตรง (ถ้าเป็นไปได้)

ภาคผนวก D: Test Strategy Template

ชั้น	ประเภท Test	เครื่องมือ	ครอบคลุม
Entities	Unit test	JUnit	Business rules
Use Cases	Unit test (mock)	JUnit + Mockito	Orchestration
Controllers	Unit test (mock Use Case)	JUnit + MockMvc	Request/Response
Presenters	Unit test	JUnit	Formatting
Gateways	Integration test	Testcontainers	Database
E2E	E2E test	Selenium/Cypress	Critical flows

ภาคผนวก E: Technology Migration Checklist

- [ ] มี interface สำหรับ infrastructure
- [ ] Business logic ไม่ผูกกับเทคโนโลยี
- [ ] Entity ไม่มี annotation ของ framework
- [ ] Use Case ไม่เรียก database โดยตรง
- [ ] มี test ครอบคลุม business logic
- [ ] มี migration plan
- [ ] มี rollback plan
- [ ] มี monitoring หลัง migration

ภาคผนวก F: Code Structure Template

```
src/
├── domain/                    # Entities
│   ├── model/
│   │   ├── Order.java
│   │   ├── OrderItem.java
│   │   └── Money.java
│   └── event/
│       └── OrderPlaced.java
├── usecase/                   # Use Cases
│   ├── PlaceOrderUseCase.java
```

```
│   ├── PlaceOrderCommand.java
│   ├── PlaceOrderResult.java
│   └── OrderRepository.java   # interface
├── adapter/                   # Interface Adapters
│   ├── controller/
│   │   └── OrderController.java
│   ├── presenter/
│   │   └── OrderPresenter.java
│   └── gateway/
│       └── JpaOrderRepository.java
└── infrastructure/            # Frameworks & Drivers
    ├── Application.java
    ├── config/
    │   └── AppConfig.java
    └── persistence/
        └── OrderJpaEntity.java
```

ภาคผนวก G: Glossary เฉพาะเล่ม 4

คำ	ความหมายโดยย่อ
Clean Architecture	สถาปัตยกรรมที่แยก business logic จากเทคโนโลยี
Dependency Rule	Source code dependencies ชี้เข้าหาศูนย์กลาง
Entities	Business rules ระดับองค์กร
Use Cases	Application-specific business rules
Interface Adapters	แปลงข้อมูลระหว่าง Use Cases กับ Frameworks
Frameworks & Drivers	เทคโนโลยี (database, web, UI)
Dependency Inversion	ชั้นในกำหนด interface, ชั้นนอก implement
Dependency Injection	Object ไม่สร้าง dependency เอง
Inversion of Control	การควบคุมถูก invert
Repository	Abstraction สำหรับเข้าถึง aggregate
Gateway	Implementation ของ repository interface
Presenter	แปลง Use Case output เป็น view model

คำ	ความหมายโดยย่อ
Controller	รับ request, เรียก Use Case
Mapper	แปลง Entity ↔ DTO/Database
Test Pyramid	unit > integration > e2e
Ports & Adapters	ชื่อเดิมของ Hexagonal Architecture
Onion Architecture	สถาปัตยกรรมชั้น concentric
Use Case	ลำดับการทำงานของ application

เกณฑ์ประเมินความสำเร็จของเล่ม

เกณฑ์	ตัวชี้วัด	น้ำหนัก
ความเข้าใจ	อธิบาย Dependency Rule และสี่ชั้นได้	20%
การออกแบบ	มี Use Case Template และ Code Structure	25%
การทดสอบ	มี Test Strategy	20%
การแยกชั้น	มี Layer Dependency Checklist	20%
การส่งต่อ	เข้าใจว่าบทใดเชื่อมไปเล่ม 5	15%

สรุปเล่ม 4

เล่ม 4 ทำหน้าที่เป็นคู่มือสำหรับการออกแบบซอฟต์แวร์ที่แยก business logic ออกจากเทคโนโลยีด้วย Clean Architecture ผู้อ่านจะได้:

1. **ความเข้าใจหลักการ** — Clean Architecture คืออะไร, Dependency Rule
2. **สี่ชั้น** — Entities, Use Cases, Interface Adapters, Frameworks & Drivers
3. **การแยก business logic** — จาก database, framework, UI
4. **DI และ IoC** — การใช้ Dependency Injection
5. **การทดสอบ** — test strategy สำหรับแต่ละชั้น

6. **การเปลี่ยนเทคโนโลยี** — โดยกระทบน้อยที่สุด
7. **ข้อดี/ข้อเสีย** — trade-offs อย่างครบถ้วน
8. **กรณีศึกษา** — ตัวอย่างการออกแบบระบบจริง

เมื่ออ่านจบ ผู้อ่านควรมีความพร้อมในการออกแบบระบบที่ทดสอบง่าย เปลี่ยนเทคโนโลยีได้ง่าย และ maintain **ได้ในระยะยาว**
